

PROJECT DOCUMENTATION

PulsePass

Booking and Event Management Web Application

A full-stack Next.js and MySQL platform for organisers, attendees, and administrators.

Document Type	Software Web Development project documentation
Application Name	PulsePass
Technology Stack	Next.js 15, React 19, TypeScript, MySQL, Zod
Prepared On	09 May 2026
Primary Purpose	Demonstrate project scope, architecture, features, and setup

This report summarises the system design, user workflows, database model, setup process, and assignment-oriented feature coverage of the PulsePass platform.

Executive Summary

PulsePass is a responsive event booking platform designed to support three core actors: organisers who publish and manage events, attendees who discover and reserve seats, and administrators who supervise users, platform activity, and support tickets. The application combines server-rendered and interactive user experiences with a MySQL-backed data model and validation-first server actions.

The project demonstrates practical full-stack development concerns including session-based access control, role-based routing, event lifecycle management,

booking concurrency protection, database transactions, responsive interface design, and maintainable modular code organisation.

Project Objectives

- Provide a public storefront for browsing upcoming events with search and filter tools.
- Allow organisers to create, edit, monitor, and remove event listings from a dedicated dashboard.
- Allow attendees to register, log in, manage bookings, and maintain an active e-ticket card.
- Allow admins to manage users, delete platform events, and resolve support tickets.
- Demonstrate secure data handling, strong validation, and clean project structure for assessment purposes.

System Overview

The application is built with the Next.js App Router and uses server components for route-level data composition, React client components for interactive forms and filters, and server actions to process authenticated mutations. Data is stored in MySQL tables for users, sessions, events, bookings, e-ticket cards, and contact tickets.

Architecture Summary

Area	Details
Frontend	Next.js App Router pages, reusable UI components, responsive CSS, client-side filters, and action-driven forms.
Backend	Server actions for login, registration, events, bookings, admin actions, contact submissions, and e-ticket operations.
Database	MySQL schema with relational constraints, indexes, and transactional booking logic.
Security	Scrypt password hashing, HTTP-only session cookies, SHA-256 token hashing, and role-based route protection.
Validation	Zod schemas validate registration, login, events, bookings, user changes, and support messages.

User Roles and Core Workflows

Role Responsibilities

Area	Details
Organiser	Creates events, edits event details, monitors bookings, and removes their own listings.
Attendee	Searches the marketplace, books seats, manages reservations, and maintains an e-ticket card.
Admin	Manages user roles, deletes platform events when necessary, and resolves open contact tickets.

Typical End-to-End Flow

1. A visitor browses public event listings through the marketplace page.
2. A new user creates an organiser or attendee account and signs in.
3. Organisers publish event listings with structured details such as city, venue, category, pricing, and capacity.
4. Attendees open an event detail page, confirm a valid e-ticket card, and reserve up to six seats in one booking.

5. Admins monitor platform activity from the control dashboard and can resolve tickets or remove problematic data.

Key Features

Feature Coverage

Area	Details
Login & Registration	Registration, login, logout, password hashing, and session creation through server actions.
Role-Based Access	Route protection redirects users to the correct dashboard based on their assigned role.
Event Management	Create, update, cancel, preview, and delete event listings from organiser and admin workflows.
Booking Management	Attendees can create and cancel bookings with duplicate booking and capacity checks.
Support Workflow	Contact form submissions are stored in the database and can be resolved from the admin dashboard.
Marketplace UX	Homepage includes live search plus city and category filtering for event discovery.
Spotlight Page	A dedicated page shows the next scheduled event based on date, with booking progress, seat stats, and supporting event recommendations.
FAQ Page	A structured help centre with accordion-style questions covering booking, accounts, events, and payments.
About Page	A static page describing the platform mission, how each role works, and team background information.
E-Ticket QR Code	Each confirmed ticket displays a QR code generated entirely in code using a custom Reed-Solomon error correction implementation — no external library.
Responsive UI	Layouts adapt across dashboards, cards, tables, and public-facing pages.

Database Design

The relational data model supports the full booking lifecycle. Users can own events, attend bookings, hold sessions, and receive e-ticket cards. Events and bookings are linked with foreign keys, while contact tickets provide a lightweight support workflow for admin review.

Main Tables

Area	Details
users	Stores identity, email, password hash, role, ownership flag, and account creation date.
sessions	Stores hashed session tokens, expiry timestamps, and links sessions to users.
events	Stores organiser ownership, event metadata, timing, pricing, capacity, descriptions, and status.
e_ticket_cards	Stores attendee-linked card status, issue date, and expiry date.
bookings	Stores booking code, event link, attendee link, seats, totals, and booking state.
contact_tickets	Stores sender details, message content, status, and submission timestamp.

Validation, Security, and Reliability

- Passwords are hashed with bcrypt and session tokens are stored in the database as SHA-256 hashes.
- Session cookies are HTTP-only and use expiry-based server validation.
- Role checks are enforced before protected dashboard actions are executed.
- Zod schemas enforce field-level validation for all major forms and action payloads.
- Booking creation uses a transaction with SELECT FOR UPDATE to reduce race conditions around seat allocation.

- Confirmed bookings are cancelled automatically when an organiser or admin deletes an event.
- Admin event deletion and organiser event deletion are wrapped in database transactions so a partial failure cannot leave bookings in an inconsistent state.
- The QR code on each e-ticket is generated without any external library, using a custom Reed-Solomon error correction implementation.
- User-submitted content is rendered through standard Next.js JSX output, which escapes HTML by default and helps mitigate XSS attacks.

Project Structure

Repository Layout

Area	Details
app/	Route files for public pages, dashboards, and event detail pages.
actions/	Server actions for login, registration, events, bookings, e-ticket handling, admin tasks, and contact logic.
components/	Reusable layout, form, and UI building blocks.
lib/	Session helpers, database utilities, formatting helpers, validation schemas, and typed query helpers.
database/	Schema and migration SQL for the application data model.
scripts/	Database seed, migration scripts, and supporting automation.

Setup and Local Usage

1. Install project dependencies with `npm install`.
2. Copy `.env.local.example` to `.env.local` and provide the local MySQL configuration.
3. Start the MySQL service on the development machine.
4. Seed the database with `npm run seed`.
5. Start the local development server with `npm run dev`.

6. Use `npm run typecheck` to generate route types and verify TypeScript correctness.

Demo Accounts

Login Reference

Account	Role	Password
admin@pulsepass.local	Admin (owner)	Passo123!
organiser@pulsepass.local	Organiser	Passo123!
attendee@pulsepass.local	Attendee	Passo123!
ava@pulsepass.local	Attendee	Passo123!
noah@pulsepass.local	Attendee	Passo123!
sofia@pulsepass.local	Attendee	Passo123!
ethan@pulsepass.local	Attendee	Passo123!

Assignment-Oriented Coverage

Requirement Mapping

Area	Details
User registration and login	Implemented through dedicated register and login flows backed by server-side validation.
Role-based access control	Protected dashboards and actions route users according to admin, organiser, or attendee roles.
Event CRUD	Organisers can create, edit, and delete events; admins can remove any event.
Booking management	Attendees can create and cancel bookings while capacity is enforced transactionally.
User administration	Admins can update roles and delete user accounts subject to ownership rules.
Contact ticket workflow	Public contact submissions are stored and can be resolved from the admin dashboard.
Server-side validation	Zod schemas cover forms for login, registration, events, bookings, admin actions, and contact tickets.
Session management	Session cookies and hashed tokens are used to maintain authenticated access with 7-day expiry.
Modular structure	The repo separates pages, actions, components, data utilities, and schema logic.
Responsive design	Grid and card-based interfaces adapt across the public site and dashboards.

Contributions

In this section, I summarise how we divided the work across the project. I based this breakdown on the recorded GitHub and repository commit history together with the contributor name clarifications we applied in the documentation. The percentages are approximate and are intended to reflect each person's main area of contribution as fairly as possible.

Division of Labour

Area	Details
Ege Gucveren	I led the project foundation and core architecture. My recorded GitHub work includes creating the initial full-stack baseline, setting up the main Next.js structure, shared styling system, session utilities, organiser and attendee dashboards, event CRUD foundation, reusable UI components, database schema, validation setup, seed workflow, and later refinements to homepage queries, event visuals, media layout, configuration, and general cleanup.
Muhammed Furkan Erdem	Focused on attendee-facing event discovery and booking experience. Work includes custom search result cards, event detail layout improvements, smart booking calls to action, the yearly e-ticket card flow, and QR ticket support built with a custom Reed-Solomon error correction implementation without any external library.
Eyupcan Dinsever	My work focused on fixing bugs and making the backend more reliable. I built the contact form and the admin ticket panel so admins can see and resolve user messages. I also added ownership rules so regular admins cannot change or delete other admin accounts. On the bug-fix side, I found that deleting an event did not cancel the bookings linked to it. I fixed this for both the organiser and the admin delete flows. I also wrapped related database operations inside transactions so if one step fails, the whole action rolls back safely. Another issue was a query that was hiding errors and fixed it to only ignore the expected case and throw everything else.
Berkay Yalmaz	Focused on user-facing content and supporting pages. Work includes the FAQ help centre with accordion questions across four topic groups, the Spotlight event page with live booking progress and seat statistics, refund policy updates, and supporting styling and query changes.

Known Notes

- The project relies on an active MySQL instance and environment configuration before data-backed features can be demonstrated.
- Production builds may depend on network availability when external Google fonts are fetched during build-time.
- The documentation in this report is based on the current repository state and its included setup instructions.

Conclusion

PulsePass demonstrates a complete multi-role event management workflow with secure session-based access, structured server-side validation, transactional booking logic, and a clean component-based interface. The project balances technical implementation concerns with user-focused flows, making it a strong example of a full-stack software web development submission.